AI Foundations I: Setup & Tools

A secure, ready-to-use AI research environment, configured with a setup wizard.

Nicholas Warren

Lead, Artificial Intelligence & Machine Learning program, DISSC

nick.warren@yale.edu

Session objectives

By the end of this session you will be able to:

- Set up a secure, containerized research environment with Docker and VS Code
- Navigate the VS Code interface and integrated terminal
- Understand how the Docker container, VS Code, and the AI service connect
- Verify that your environment is configured and ready to use

Why this matters

Research workflows in R involve repetitive, time-consuming tasks: cleaning data, debugging cryptic error messages, writing routine code, and iterating on visualizations. Claude Code can handle these so you can focus on the research questions.

- **Hours to minutes.** Tasks like reformatting data, writing plot code, or adding robust standard errors can be done by typing a single request.
- **Lower the barrier.** You don't need to memorize package syntax. Describe what you want in plain English and refine from there.
- **Stay in your editor.** No switching to a web browser. Claude works alongside your code and can read your files directly.

Setup

These instructions cover macOS and Windows. Follow only the slides for your platform.

1. Install Node.js

Node.js is needed to run the setup wizard.

macOS, recommended via Homebrew:

```
brew install node
```

Windows, recommended via **winget** (preinstalled on Windows 10/11):

```
winget install OpenJS.NodeJS.LTS
```

> **No package manager?** Download the **LTS** installer from https://nodejs.org/ instead. On Windows, keep the default options (including "Add to PATH").

1. Install Node.js (verify)

After installation, open a **new Terminal** (macOS) or **new PowerShell** (Windows) window and verify:

```
node --version
```

You should see a version number like `v22.x.x`.

> Opening a *new* window matters: `node` won't be recognized in windows that were already open when you installed it.

2. Request an API key

An **API key** is like a password that lets your computer connect to Claude. Email Nicholas Warren at DISSC (dissc@yale.edu) to request one. **Request this before the workshop;** you'll need the key in Step 3. This key will expire after the workshop. For long-term access, contact dissc@yale.edu.

> **Treat your API key like a password.** Do not share it or post it anywhere.

3. Run the setup wizard

Open **Terminal** (macOS) or **PowerShell** (Windows) and move to the folder where you want your project to live:

```
cd ~/Desktop
```

Then run the wizard (replace `my-workshop` with any name you like):

```
npm create @yale-dissc/agent-sandbox@latest my-workshop
```

> `~` works in both zsh (macOS) and PowerShell (Windows). The wizard creates a folder with your chosen name in the current directory.

3. Run the setup wizard (dependency check)

The wizard checks for Git, Docker Desktop, VS Code, and the Dev Containers extension:

- For **Git, Docker, or VS Code**: the wizard opens the download page in your browser. Install manually, then press Enter to continue.
- For the **Dev Containers extension**: the wizard offers to install it for you. Accept the default (**Yes**) and it installs automatically.

3. Run the setup wizard (dependency notes)

If the wizard gets stuck on a missing dependency, one of these usually fixes it:

- **Windows — Docker:** Requires WSL 2. Run `wsl --install` in **PowerShell as Administrator**, restart, then re-run the wizard.
- **Windows — VS Code:** Reinstall with **"Add to PATH"** ticked, or restart your terminal.
- **macOS — VS Code:** In VS Code, `Cmd+Shift+P` > **Shell Command: Install 'code' command in PATH**.

3. Run the setup wizard (authentication)

For auth mode, **arrow down** to **Anthropic via AWS Bedrock** (Claude.ai is default) and press Enter. Then:

- **AWS region:** keep the default (`us-east-1`)
- **Bedrock bearer token:** paste the API key from Step 2
- **Default Opus model:** keep the default (`anthropic.claude-opus-4-7`)

> The wizard previews the changes and backs up your existing shell config before writing. Type `Y` to confirm.

3. Run the setup wizard (languages)

At the language prompt, press the **spacebar** to select **R + tidyverse + languageserver**, then press Enter. Nothing is selected by default — if you skip this step, R won't be installed.

> **First build time with R:** 15–30 minutes.

3. Run the setup wizard (finish)

The wizard then:

- Downloads the sandbox template into your project folder
- Edits the container configuration to match your language selection
- Asks about Git and GitHub: **choose "No"** for the workshop (you'll add workshop files before initializing git in Step 4)
- Asks "Open the project in VS Code now?": **choose "No"** (VS Code would immediately prompt to reopen in a container, but your `workspace/` is still empty)

4. Add workshop files (macOS)

Now add the workshop materials to the empty `workspace/` folder the wizard created:

- Download the workshop files from https://github.com/DISSC-yale/ai-workshops (green **Code** button > **Download ZIP**, then double-click to unzip)
- Open the `ai-foundations-ii/vs-code-and-r/code/` folder
- **Important:** Some folders start with a dot (`.claude/` , `.vscode/`) and may be hidden. Press `Cmd+Shift+.` in Finder to show them.
- Select everything inside it and copy it into your project's `workspace/` folder
- Also copy the `ai-foundations-ii/vs-code-and-r/prompts/` folder into `workspace/`

4. Add workshop files (Windows)

Now add the workshop materials to the empty `workspace\` folder the wizard created:

- Download the workshop files from https://github.com/DISSC-yale/ai-workshops (green **Code** button > **Download ZIP**, then right-click > **Extract All**)
- Open the `ai-foundations-ii\vs-code-and-r\code\` folder
- **Important:** Some folders start with a dot (`.claude\` , `.vscode\`) and may be hidden. In File Explorer, click **View** > **Show** > **Hidden items**.
- Select everything inside it and copy it into your project's `workspace\` folder
- Also copy the `ai-foundations-ii\vs-code-and-r\prompts\` folder into `workspace\`

5. Open the project in VS Code

Open VS Code, go to **File > Open Folder**, and select the project folder the wizard created (the one containing `workspace/` and `.devcontainer/`, not `workspace/` itself).

When VS Code prompts **"Reopen in Container?"**, click **Yes**.

> If the prompt doesn't appear, press `Cmd+Shift+P` (macOS) or `Ctrl+Shift+P` (Windows), type `Reopen in Container`, and select it.

5. Open the project in VS Code (first build)

The first build takes **15–30 minutes** because it installs and compiles R packages. Subsequent opens are fast (a few seconds). A log will appear showing setup progress; you can let it run in the background.

Once it finishes, close that log tab and open a fresh terminal inside VS Code with `Ctrl+` (that's the backtick key, usually above Tab).

6. Verify your extensions

You're now working inside the container. Because you toggled **R** in the language prompt, the wizard pre-installed R, the `tidyverse` / `languageserver` R packages, and the **R + Quarto VS Code extensions** as part of the container build.

Confirm the extensions are there:

- Click the **Extensions** icon in the left sidebar (four small squares; or `Cmd+Shift+X` / `Ctrl+Shift+X`)
- Under **Installed**, you should see **R** (REditorSupport) and **Quarto**

> If either is missing, search for it and click **Install** manually.

7. Verify your setup

In the container's terminal, start Claude Code:

```
claude
```

Run `/model` to confirm Claude is using the Bedrock-backed Opus model. Then ask a test question: `What is 2 + 2?`

You should see Claude respond with an answer. Type `/exit` to quit Claude Code.

> **If it doesn't work:** Make sure Docker Desktop is running. If VS Code was already open when the wizard saved your credentials, fully quit and relaunch it so it picks up the new environment variables.

Orientation

Now that your environment is set up, let's understand the tools you just installed.

What is Docker?

Docker creates a self-contained workspace on your computer, like a virtual computer within your computer. This workspace is called a **container**.

Why use it?

- **No installation headaches.** R, Python, and Claude Code are pre-configured and ready to go.
- **Consistent environment.** Everyone runs the same container, so there are no "it works on my machine but not yours" problems.
- **Safe to experiment.** The container is isolated from the rest of your computer. Claude can only access files in the `workspace/` folder and cannot touch your personal files, email, or anything else on your machine.

What is the terminal?

The **terminal** is a text-based interface for your computer. Instead of clicking icons, you type commands. Think of it like the R console, but for your entire machine.

Concept	What it means
Shell	The program that interprets your commands (you don't need to worry about which one you have)
Command	A text instruction like `ls` (list files) or `cd` (change folder)
Working directory	The folder your terminal is currently "in"

Essential terminal commands

These are the commands you'll use most often:

Command	What it does	Example
`ls`	List files in the current folder	`ls data/`
`cd`	Change directory (move to another folder)	`cd ~/Documents`
`pwd`	Print the current working directory	`pwd`
`cat`	Print the contents of a file	`cat data.csv`
`claude`	Start Claude Code	`claude`

> You don't need to memorize these. Claude Code can run them for you.

What is VS Code?

Visual Studio Code (VS Code) is a free code editor made by Microsoft. Think of it as a modern alternative to RStudio that works with many languages.

Key differences from RStudio:

- VS Code supports **any programming language** (R, Python, Stata, etc.) through add-ons called **extensions**
- It has a built-in **terminal** where you can type commands (like the one you used during setup)
- It is the most widely used code editor in the world, with a large library of extensions

Navigating the VS Code interface

Area	Location	What it does
Activity Bar	Far left (icons)	Switch between Explorer, Search, Extensions
Side Bar	Left panel	File tree, search results, or extensions
Editor	Center	View and edit files. Run code with `Cmd/Ctrl + Enter`
Panel	Bottom	Terminal and R Terminal. Toggle with `Ctrl + ` `

Key actions in VS Code

Action	How
Find your files	Click the top Activity Bar icon to open the **Explorer**
Open a terminal	`Ctrl + ` ` or `Terminal > New Terminal`
Install extensions	`Cmd/Ctrl + Shift + X` to open the Extensions view
Command Palette	`Cmd/Ctrl + Shift + P` (universal search for any command or setting)

VS Code for RStudio users

RStudio	VS Code equivalent
Console	**R Terminal** (bottom Panel)
Source editor	**Editor** (center). Run lines with `Cmd/Ctrl + Enter`
Files pane	**Explorer** (top icon in Activity Bar)
Environment pane	**R: Workspace** (R icon in Activity Bar)
Plots pane	Inline in the notebook (`.qmd` files, similar to R Markdown)
Help	Hover over a function, or `?fn` in R Terminal

How it works

In this workshop you use **VS Code** as your editor and **Claude Code** as your AI assistant, all running inside a **Docker container**. Claude Code is a tool made by **Anthropic** (the company behind Claude) that lets you interact with Claude directly from the terminal.

When you type a message to Claude Code, it is sent to **Claude Opus 4.7** (the most capable Claude model) through **AWS Bedrock**, Amazon's AI service, running on Yale's AWS account.

Data security

- **Your data is not used for training.** AWS Bedrock does not use your prompts or outputs to train any models.
- **Your data is not shared.** Model providers (including Anthropic) do not have access to your inputs, outputs, or logs.
- **Enterprise compliance.** AWS Bedrock is HIPAA eligible, SOC and ISO certified, and GDPR compliant.

In short: you get the full power of Claude without your research data leaving Yale's AWS environment or being used to train future models.

Key concepts

Tool	What it is
Docker	Creates the self-contained workshop environment.
VS Code	Microsoft's free code editor. Your main working environment.
Dev Containers	VS Code extension that connects VS Code to Docker containers.
Setup wizard	Checks your machine, configures credentials, downloads the sandbox.
AWS Bedrock	Amazon's AI service. Routes Claude requests via Yale's AWS.
API key	Password-like key that proves you're authorized to use Claude.

One project per sandbox

Each sandbox is meant for a single project. Claude operates on the entire `workspace/` folder, so mixing unrelated files risks unintended changes across projects.

To start a new project, run the setup wizard again with a different name:

```
npm create @yale-dissc/agent-sandbox@latest my-next-project
```

Each project gets its own folder, container, and `workspace/`.